CS 601.471/671 NLP: Self-supervised Models

Homework 4: Neural Language Modeling + Fixed-Window LMs

For homework deadline and collaboration policy, check the calendar on the course website[1]

Name: Dontavious Jennings

Collaborators, if any: _____

Sources used for your homework, if any: _____

This assignment focuses on language modeling, while continuing to build up on our prior knowledge of neural networks. We will review several aspects about training neural nets and also extend it to modeling sequences in language.

Homework goals: After completing this homework, you should be comfortable with:

- thinking more deeply about training neural networks; debugging your neural network
- getting more engaged in using PyTorch for training NNs
- training your first neural LM

Concepts, intuitions and big picture

Multiple-choice questions.

1. Select the sentence that best describes the terms “model”, “architecture”, and “weights”.
   - Model and weights are the same; they form a succession of mathematical functions to build an architecture.
   - An architecture is a succession of mathematical functions to build a model and its weights are those functions parameters.

An architecture is a succession of mathematical functions to build a model and its weights are those functions parameters.

2. During forward propagation, in the forward function for a layer l you need to know what is the activation function in a layer (Sigmoid, tanh, ReLU, etc.). During Backpropagation, the corresponding backward function does not need to know the

[1] https://self-supervised.cs.jhu.edu/sp2024/

activation function for layer l since the gradient does not depends on it.

- True
- False

False. The backward function needs the derivative of the activation, and that derivative depends on which activation was used (for example, sigmoid' = s(1 - s) while ReLU' = 1{z > 0}).

3. You have built a network using the tanh activation for all the hidden units. You initialize the weights to relative large values, using `randn(..,..)*1000`. What will happen?
   - It doesn't matter. So long as you initialize the weights randomly gradient descent is not affected by whether the weights are large or small.
   - This will cause the inputs of the tanh to also be very large, thus causing gradients to also become large. You therefore have to set the learning rate to be very small to prevent divergence; this will slow down learning.
   - This will cause the inputs of the tanh to also be very large, causing the units to be "highly activated" and thus speed up learning compared to if the weights had to start from small values.
   - This will cause the inputs of the tanh to also be very large, thus causing gradients to be close to zero. The optimization algorithm will thus become slow.

This will cause the inputs of the tanh to also be very large, thus causing gradients to be close to zero. The optimization algorithm will thus become slow.

4. What is the "cache" used for in our implementation of forward propagation and backward propagation? - It is used to cache the intermediate values of the cost function during training.
   - We use it to pass variables computed during forward propagation to the corresponding backward propagation step. It contains useful values for backward propagation to compute derivatives.
   - It is used to keep track of the hyperparameters that we are searching over, to speed up computation.
   - We use it to pass variables computed during backward propagation to the corresponding forward propagation step. It contains useful values for forward propagation to compute activations.

We use it to pass variables computed during forward propagation to the corresponding backward propagation step. It contains useful values for backward propagation to compute derivatives.

5. Among the following, which ones are "hyperparameters"? (Check all that apply.) - size of the hidden layers.

- learning rate
- number of iterations
- number of layers in the neural network

All four: size of the hidden layers, learning rate, number of iterations, and number of layers in the neural network.

6. True/False? Vectorization allows you to compute forward propagation in an L-layer neural network without an explicit for-loop (or any other explicit iterative loop) over the layers $l = 1,2, \ldots, L$. ▫ True ▫ False

False. Vectorization removes the loops over examples and over units within a layer, but each layer needs the previous layer's output, so you still need a loop over the layers.

7. True or false? A language model usually does not need labels annotated by humans for its pretraining. ▫ True ▫ False

True.

8. What is the order of the language modeling pipeline?
   ▫ First, the model, which handles text and returns raw predictions. The tokenizer then makes sense of these predictions and converts them back to text when needed.
   ▫ First, the tokenizer, which handles text and returns IDs. The model handles these IDs and outputs a prediction, which can be some text.
   ▫ The tokenizer handles text and returns IDs. The model handles these IDs and outputs a prediction. The tokenizer can then be used once again to convert these predictions back to some text.

The tokenizer handles text and returns IDs. The model handles these IDs and outputs a prediction. The tokenizer can then be used once again to convert these predictions back to some text.

Short answer questions

1. Look at the definition of [Stochastic] Gradient Descent in PyTorch: https://pytorch.org/docs/stable/generated/torch.optim.SGD.html. You will see that this is a bit more complex than what we have seen in the class. Let's understand a few nuances here.

   a. Notice the `maximize` parameter which controls whether we are running a maximization or minimization. In the algorithm the effect of this parameter is shown as essentially a change in the sign of the gradients:

$$\textbf{if } maximize$$
$$\theta_t \leftarrow \theta_{t-1} + \gamma g_t$$
$$\textbf{else}$$
$$\theta_t \leftarrow \theta_{t-1} - \gamma g_t$$

How do you think this change leads to the desired outcome (maximization vs minimization)?

The gradient points in the direction where the objective increases fastest. Subtracting gamma * g_t moves the parameters downhill, so the objective goes down (minimization), while adding gamma * g_t moves them uphill, so the objective goes up (maximization). Flipping the sign of the gradient is the same as minimizing the negative of the objective.

b. The next set of parameters `momentum`, `dampening`, `weight_decay`. What do you think the impact of these parameters are? Read the documentations and interpret their roles.

Momentum keeps a running buffer of past gradients, b_t = mu * b_(t-1) + (1 - tau) * g_t, and steps in the direction of that buffer instead of the raw gradient. This smooths out noisy mini-batch gradients and keeps the updates moving through flat regions and narrow valleys, so training usually converges faster.

Dampening (tau) controls how much the current gradient adds to the momentum buffer. With dampening = 0 the full gradient is added, and larger values make each new gradient count less.

Weight decay adds lambda * theta to the gradient before the update, which is the same as adding an L2 penalty on the weights. It pulls the weights toward zero a little on every step, which acts as regularization and helps against overfitting.

2. What are few benefits to using Fixed-Window-LM over n-grams language models? (limit your answer to less then 5 sentences).

A fixed-window LM represents words with embeddings, so similar words share information and the model can give sensible probabilities to sequences it never saw in training, while an n-gram model gives them zero probability unless it uses smoothing or backoff. It also does not have to store a count for every n-gram, so the model size does not blow up as the window gets longer. The parameters are shared across all contexts, which makes it generalize better from the same amount of data.

3. When searching over hyperparameters, a typical recommendation is to do random search rather than a systematic grid search. Why do you think that is the case? (less than 2 sentences).

Usually only a few hyperparameters really matter, and random search tries many more distinct values of each one for the same number of runs, while a grid wastes runs repeating the same few values of the important ones.

4. Remember the normalization layer that we saw during the class. Here is the corresponding PyTorch page: https://pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html. In the normalization formula, why do we use epsilon ϵ in the denominator?

Epsilon keeps the denominator sqrt(Var[x] + epsilon) from being zero or extremely small. If all the features in a vector are the same, the variance is 0 and we would divide by 0; with a tiny variance the output would blow up. Adding epsilon keeps the computation numerically stable.

Softmax Smackdown: Squishing the Competition

Softmax gradient

You might remember in the midterm exam that we saw a neural network with a Softmax and a cross-entropy loss:

$$\hat{\mathrm{y}} = \mathrm{Softmax}(\mathrm{h}),\ J = \mathrm{CE}(\mathbf{y}, \hat{\mathbf{y}}), \qquad \mathrm{h}, \mathbf{y}, \hat{\mathbf{y}} \in \mathbb{R}^d.$$

Basically here $\hat{\mathbf{y}}$ is a d-dimensional probability distribution over d possible options (i.e. $\sum_i \mathbf{y}_i = 1$ and $\forall i: \mathbf{y}_i \in [0,1]$). $\mathbf{y}$ is a d-dimensional one-hot vector, i.e., all of these values are zeros except one that corresponds to the correct label.

Here we want to prove the hint that was provided in the exam, which was:

$$\nabla_{\mathbf{h}} J = \hat{\mathbf{y}} - \mathbf{y}.$$

Prove the above statement. In your proof, use the statement that you proved in HW3 about gradient of Softmax function: $\frac{d\hat{\mathrm{y}}_i}{d\mathrm{h}_j} = \hat{\mathrm{y}}_i \left(\delta_{ij} - \hat{\mathrm{y}}_j\right)$, where δ_{ij} is the Kronecker delta function and $\hat{\mathrm{y}}_i$ is the value in the i-th index of $\hat{\mathrm{y}}$.

With cross-entropy J = -sum_i y_i log(y_hat_i), take the derivative with respect to h_j:

dJ/dh_j = -sum_i y_i * (1 / y_hat_i) * (d y_hat_i / d h_j)

Using the HW3 result d y_hat_i / d h_j = y_hat_i (delta_ij - y_hat_j):

dJ/dh_j = -sum_i y_i * (1 / y_hat_i) * y_hat_i (delta_ij - y_hat_j) = -sum_i y_i (delta_ij - y_hat_j) = -y_j + y_hat_j * sum_i y_i

Since y is one-hot, sum_i y_i = 1, so dJ/dh_j = y_hat_j - y_j. Stacking this for every j gives the gradient vector: grad_h J = y_hat - y.

Softmax temperature

Remember the softmax function, $\sigma(\mathbf{z})$? Here we will add a *temperature* parameter $\tau \in \mathbb{R}^+$ to this function:

$$\text{Softmax: } \sigma(\mathbf{z};\tau)_i = \frac{e^{z_i/\tau}}{\sum_{j=1}^{K} e^{z_j/\tau}} \quad \text{for } i = 1, \ldots, K$$

Show the following:

1. In the limit as temperature goes to zero $\tau \to 0$, softmax becomes the same as greedy action selection, `argmax`.

Let z_max = max_j z_j and divide the top and bottom by e^(z_max / tau): sigma(z; tau)_i = e^((z_i - z_max) / tau) / sum_j e^((z_j - z_max) / tau).

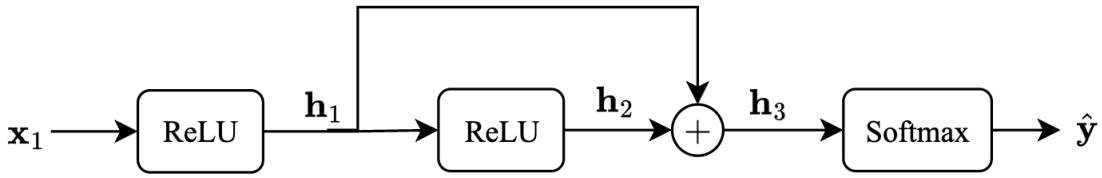
Every exponent (z_j - z_max) / tau is <= 0. As tau -> 0+, the exponent goes to -infinity for any j with z_j < z_max, so those terms go to 0, while for the entries equal to z_max the exponent stays 0 and the term stays 1. With a single maximum, sigma_i -> 1 for i = argmax z and 0 for every other i, which is exactly greedy argmax selection. (If m entries tie for the max, each gets 1/m.)

2. In the limit as temperature goes to infinity $\tau \to +\infty$, softmax gives equiprobable selection among all actions.

As tau -> +infinity, z_i / tau -> 0 for every i because the z_i are fixed finite numbers, so every e^(z_i / tau) -> e^0 = 1. The softmax becomes sigma_i -> 1 / sum_j 1 = 1/K for every i, which is an equiprobable choice among all K actions.

Backprop Through Residual Connections

As you know, When neural networks become very deep (i.e. have many layers), they become difficult to train due to the vanishing gradient problem – as the gradient is back-propagated through many layers, repeated multiplication can make the gradient extremely small, so that performance plateaus or even degrades. An effective approach is to add skip connections that skip one or more layers. See the provided network.



$$\mathbf{x} \in \mathbb{R}^d,\ \mathbf{W}_{1,2} \in \mathbb{R}^{d \times d},\ \hat{\mathbf{y}} \in \mathbb{R}^d$$

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x}_1,\ \mathbf{h}_1 = \mathrm{ReLU}(\mathbf{z}_1),$$

$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{h}_1,\ \mathbf{h}_2 = \mathrm{ReLU}(\mathbf{z}_2),$$

$$\mathbf{h}_3 = \mathbf{h}_1 + \mathbf{h}_2,\ \hat{\mathbf{y}} = \mathrm{Softmax}(\mathbf{h}_3),\ J = \mathrm{CE}(\mathbf{y}, \hat{\mathbf{y}})$$

1. You have seen this in the quiz, but lets try again. Prove that: $\frac{\partial}{\partial z_i}\mathrm{ReLU}(z) = 1\{z_i > 0\}$ where $1\{x > 0\} = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$. More generally, $\nabla_z \mathrm{ReLU}(z) = \mathrm{diag}(1\{z > 0\})$ where $1\{.\}$ is applied per each dimension and diag(.) turns a vector into a diagonal matrix.

ReLU(z)_i = max(0, z_i). If z_i > 0, ReLU(z)_i = z_i, so the derivative is 1. If z_i < 0, ReLU(z)_i = 0, so the derivative is 0. At z_i = 0 the function has a kink and is not differentiable, so we use 0 there by convention, which gives d/dz_i ReLU(z) = 1{z_i > 0}.

For the full Jacobian, ReLU(z)_i only depends on z_i, so d ReLU(z)_i / d z_k = 0 for every k != i. The off-diagonal entries are all 0 and the diagonal entries are 1{z_i > 0}, so grad_z ReLU(z) = diag(1{z > 0}).

2. As you see, a single variable ($\mathbf{h}_2$ in the example) feeds into two different layers of the network. Here we want to show that the two upstream signals stemming from this node are merged as summation during Backprop. Specifically prove that:

h3 = h1 + h2, and h2 depends on h1 through z2 = W2 h1 and h2 = ReLU(z2). Differentiating the sum term by term:

dh3/dh1 = dh1/dh1 + dh2/dh1 = I + (dh2/dz2)(dz2/dh1)

The first term is the identity because h1 is added directly, and the second term is the chain rule through the ReLU branch. h1 reaches h3 along two paths, so the two upstream signals add up during backprop.

$$\frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_1} = \mathrm{I} + \frac{\partial \mathbf{h}_2}{\partial \mathbf{z}_2}\frac{\partial \mathbf{z}_2}{\partial \mathbf{h}_1}$$

3. In the given neural network your task is to **compute the gradient** $\frac{\partial J}{\partial \mathbf{x}}$. You are allowed (and highly encouraged) to use variables to represent intermediate gradients.

Hint 1: Compute these gradients in order to build up your answer: $\frac{\partial J}{\partial h_3}$, $\frac{\partial J}{\partial h_2}$, $\frac{\partial J}{\partial z_2}$, $\frac{\partial J}{\partial h_1}$, $\frac{\partial J}{\partial z_1}$, $\frac{\partial J}{\partial x}$. Show your work so we are able to give partial credit!

Hint 2: Recall that *downstream* = *upstream* * *local*.

Writing gradients as row vectors (downstream = upstream * local), with z1 = W1 x, h1 = ReLU(z1), z2 = W2 h1, h2 = ReLU(z2), h3 = h1 + h2, y_hat = Softmax(h3):

dJ/dh3 = (y_hat - y)^T, from the softmax + cross-entropy result above. Call this delta.

dJ/dh2 = dJ/dh3 * dh3/dh2 = delta * I = delta

dJ/dz2 = dJ/dh2 * diag(1{z2 > 0}) = delta * diag(1{z2 > 0})

dJ/dh1 = dJ/dh3 * dh3/dh1 = delta * (I + diag(1{z2 > 0}) W2) = delta + delta * diag(1{z2 > 0}) W2

dJ/dz1 = dJ/dh1 * diag(1{z1 > 0})

dJ/dx = dJ/dz1 * W1

Putting it together: dJ/dx = [(y_hat - y)^T (I + diag(1{z2 > 0}) W2)] diag(1{z1 > 0}) W1. As a column vector: grad_x J = W1^T (1{z1 > 0} ∘ [(y_hat - y) + W2^T (1{z2 > 0} ∘ (y_hat - y))]), where ∘ is element-wise multiplication.

4. Based on your derivations, explain why residual connections help mitigate vanishing gradients.

From dJ/dh1 = delta + delta * diag(1{z2 > 0}) W2, the gradient reaching h1 includes a direct copy of the upstream gradient through the identity path. Even if the learned branch contributes almost nothing, there is still a path for the gradient to move backward. In a deep network this makes vanishing gradients much less severe because every block has an identity path instead of forcing the gradient through only the learned transformations.

Programming

In this programming homework, we will

- implement your own subword tokenizer.
- implement fixed-window MLP language models

Skeleton Code and Structure:

The code base for this homework can be found at this GitHub repo under the `hw3` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements,

guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `tokenization.py` implements the Byte-Pair Encoding algorithm for subword tokenization.
- `mlp_lm.py` implements a fixed-window MLP-based language model on a subset of Wikipedia.
- `main.py` provides the entry point to run your implementations in both `tokenizatiton.py` and `mlp_lm.py`.
- `hw4.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a **`# TODO`** at the corresponding blank in the code.

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file

Tokenization Algorithms

All NLP systems have *at least* three main components that help machines understand natural language:

1. Tokenization
2. Embedding
3. Model architectures

Models like BERT, GPT-2 or GPT-3 all share the same components but with different architectures that distinguish one model from another. We are going to focus on the first component of an NLP pipeline which is **tokenization**. An often overlooked concept but it is a field of research in itself. Though we have SOTA algorithms for tokenization it's always a good practice to understand the evolution trail.
Here's what we'll cover:

- What is tokenization?
- Why do we need a tokenizer?

- Types of tokenization - Word, Character and Subword.
- Byte Pair Encoding Algorithm
- Other tokenization algorithms:
    - Unigram Algorithm
    - WordPiece - BERT transformer
    - SentencePiece - End-to-End tokenizer system

What is Tokenization?

Tokenization is the process of representing raw text in smaller units called tokens. These tokens can then be mapped with numbers to further feed to an NLP model.

Read and run the `full_word_tokenization_demo` in `tokenization.py` for an overly simplified example of what a tokenizer does.

This is a very simple example where we just split a text string in to “whole words” on white spaces, and we have not considered grammar, punctuation, or compound words (like “test”, “test-ify”, “test-ing”, etc.).

Problems with word tokenization:

- **Missing words in the training data**: With word tokens, your model won’t recognize the variants of words that were not part of the data on which the model was trained. So, if your model has seen ‘foot‘ and ‘ball‘ in the training data but the final text has “football”, the model won’t be able to recognize the word and it will be treated with a “<UNK>” token. Similarly, punctuations pose another problem, “let” or “let’s” will need individual tokens and it is an inefficient solution. This will **require a huge vocabulary** to make sure you’ve every variant of the word. Even if you add a **lemmatizer** to solve this problem, you’re adding an extra step in your processing pipeline.
- **Not all languages use space for separating words**: For a language like Chinese, which doesn’t use spaces for word separation, this tokenizer will fail.

Character-based tokenization

To resolve the problems associated with word-based tokenization, an alternative approach of character-by-character tokenization was tried. This solves the problem of missing words as now we are dealing with characters that can be encoded using ASCII or Unicode and it could generate embedding for any word now.

Every character, be it space, apostrophes, or colons, can now be assigned a symbol to generate a sequence of vectors. But this approach had its cons. Character-based models will treat each character and will lead to longer sequences. For a 5-word long sentence, you may need to process 30 tokens instead of 5 word-tokens.

Subword Tokenizaiton

To address the earlier issues, we could think of breaking down the words based on a set of prefixes and suffixes. For example, we can build a system that can identify subwords like "##s", "##ing", "##ify", "un##" etc., where the position of double hash "##" denotes prefix and suffixes. So, a word like "unhappily" is tokenized using subwords like "un##", "happ", and "##ily".

BPE (Byte-Pair Encoding) was originally a data compression algorithm that is used to find the best way to represent data by identifying the common byte pairs. It is now used in NLP to find the best representation of text using the least number of tokens.

Here's how it works:

1. Add a "</w>" at the end of each word to identify the end of a word and then calculate the word frequency in the text.
2. Split the word into characters and then calculate the character frequency.
3. For a predefined number of iterations (set by you), count the frequency of the consecutive tokens and merge the most frequently occurring pairings.
4. Keep iterating until you have reached the iteration limit or if you have reached the token limit.

We will now go through this algorithm step by step.
Read the `get_word_freq` function in `tokenization.py` that implements the step 1 above to preprocess each word and count its frequency.
TODOs: Read and complete the missing lines in the `get_pairs` function in `tokenization.py` to implement the step 2 above. This function takes the word frequency record returned from `get_word_freq` as input, split the words into tokens (characters at this initial step), and counts token-pairs frequency.
Hint: follow the comments in the code for specific requirements.
Read the `get_most_frequent_pair` and `merge_byte_pairs` functions in `tokenization.py` that implements the step 3 above, merge the top-1 frequent token-pairs in all the words.
In practise, after iterating over the input text for a desired number of times, we extract the resulting tokenization as our subword dictionary. For this purpose, we provide `get_subword_tokens` function in `tokenization.py`
With all these functions, we can now run one step of the BPE algorithm!
Read and run the `test_one_step_bpe.py` in `main.py` to test the BPE algorithm for one step. A correct implementation of `get_pairs` should pass all the tests.

Complete the BPE Algorithm

With the above implementations, we can complete all the steps of the BPE algorithm that iterates over the input corpus.
TODOs: Read and complete the missing lines in the `exract_bpe_subwords` function in `tokenization.py` that implements the full BPE algorithm. Once you finished, run the `test_bpe` function in `main.py`, a correct implementation of `exract_bpe_subwords` should pass all the tests.
Hint: follow the comments in the code and you can reuse the helper functions provided/you implemented above.
So as we iterate with each best pair, we merge (concatenating) the pair and you can see as we recalculate the frequency, the original character token frequency is reduced and the new paired token frequency pops up in the token dictionary.

Running BPE on a subset of Wikipedia

Now, let's run this algorithm on a larger scale. In particular, we will run on many Wikipedia paragraphs. As usual, let's use the Huggingface library to download the data (the `load_bpe_data` function).
TODOs: Read and run the `bpe_on_wikitext` in `main.py` that iterates the BPE algorithm on a subset of Wikipedia, and interprets the final subwords extracted from the Wikipedia documents. In particular, identify examples of subwords that correspond to

1. complete words
2. part of a word
3. non-English words(including other languages or common sequence special characters)

Explain why these subwords have come about. (no more than 10 sentences)

Running 2,000 BPE merges on the last 5,000 WikiText-2 paragraphs gave 2,280 subword tokens. Complete words: very common words end up as whole tokens with the end-of-word marker, like the</w>, of</w>, was</w>, and The</w>, and so do longer words that repeat a lot in this subset, like established</w>, development</w>, and article topics like Corythosaurus</w> and Thunderbirds</w>. Part of a word: pieces without </w> such as charac, gener, struc, publ, origin, and tion are stems and endings shared by many words (character, general, structure, public, original, -tion). Non-English words and special characters: single characters from other scripts like the Georgian რ, Cyrillic к, and Chinese 動, accented letters like é and ü, WikiText's @-@, @.@, and @,@ markers, and number pieces like 193 and 201.

These results follow from how BPE works. It repeatedly merges the most common adjacent pairs, so very common words can eventually become complete tokens while less common

words only get partially merged into pieces they share with other words. Characters from other languages occur less often in this subset, so many of them stay as individual characters. WikiText markers such as @-@ appear often enough to be merged, and number fragments like 193 and 201 are probably coming from repeated number and year patterns in the articles.

Additonal Readings: Other Tokenization Algorithms

WordPiece

WordPiece is the subword tokenization algorithm used for BERT, DistilBERT, and Electra. The algorithm was outlined in and is very similar to BPE. WordPiece first initializes the vocabulary to include every character present in the training data and progressively learns a given number of merge rules. In contrast to BPE, WordPiece does not choose the most frequent symbol pair, but the one that maximizes the likelihood of the training data once added to the vocabulary.

So what does this mean exactly? Referring to the previous example, maximizing the likelihood of the training data is equivalent to finding the symbol pair, whose probability is divided by the probabilities of its first symbol followed by its second symbol is the greatest among all symbol pairs. E.g. “u”, followed by “g” would have only been merged if the probability of “ug” divided by “u”, “g” would have been greater than for any other symbol pair. Intuitively, WordPiece is slightly different from BPE in that it evaluates what it *loses* by merging two symbols to ensure it’s *worth it*.

Unigram

Unigram is a subword tokenization algorithm introduced in . In contrast to BPE or WordPiece, Unigram initializes its base vocabulary to a large number of symbols and progressively trims down each symbol to obtain a smaller vocabulary. The base vocabulary could for instance correspond to all pre-tokenized words and the most common substrings. Unigram is not used directly for any of the models in the transformers, but it’s used in conjunction with SentencePiece([paragraph:sentetncepiece]).

At each training step, the Unigram algorithm defines a loss (often defined as the log-likelihood) over the training data given the current vocabulary and a unigram language model. Then, for each symbol in the vocabulary, the algorithm computes how much the overall loss would increase if the symbol was to be removed from the vocabulary. Unigram then removes p (with p usually being 10% or 20%) percent of the symbols whose loss increase is the lowest, i.e. those symbols that least affect the overall loss over the training data. This process is repeated until the vocabulary has reached the desired size. The Unigram algorithm always keeps the base characters so that any word can be tokenized.

Because Unigram is not based on merge rules (in contrast to BPE and WordPiece), the algorithm has several ways of tokenizing new text after training. As an example, if a trained

Unigram tokenizer exhibits the vocabulary:
[“b”, “g”, “h”, “n”, “p”, “s”, “u”, “ug”, “un”, “hug”],

“hugs” could be tokenized both as [“hug”, “s”], [“h”, “ug”, “s”] or [“h”, “u”, “g”, “s”]. So which one to choose? Unigram saves the probability of each token in the training corpus on top of saving the vocabulary so that the probability of each possible tokenization can be computed after training. The algorithm simply picks the most likely tokenization in practice but also offers the possibility to sample a possible tokenization according to their probabilities.

Those probabilities are defined by the loss the tokenizer is trained on. Assuming that the training data consists of the words $x_1, \ldots, x_N$ and that the set of all possible tokenizations for a word x_i is defined as $S(x_i)$, then the overall loss is defined as

$$\mathcal{L} = -\sum_{i=1}^{N} \log\left(\sum_{x \in S(x_i)} p\,(x)\right)$$

SentencePiece

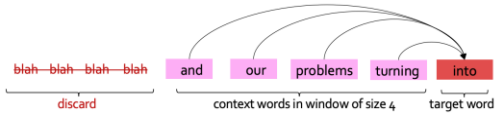
All tokenization algorithms described so far have the same problem: It is assumed that the input text uses spaces to separate words. However, not all languages use spaces to separate words. One possible solution is to use language-specific pre-tokenizers, e.g. XLM uses a specific Chinese, Japanese, and Thai pre-tokenizer). To solve this problem more generally, treats the input as a raw input stream, thus including the space in the set of characters to use. It then uses the BPE or unigram algorithm to construct the appropriate vocabulary.

The XLNetTokenizer uses SentencePiece for example, which is also why in the example earlier the “” character was included in the vocabulary. Decoding with SentencePiece is very easy since all tokens can just be concatenated and “” is replaced by a space.

All transformer models in the library that use SentencePiece use it in combination with unigram. Examples of models using SentencePiece are ALBERT XLNet,Marian, and T5.

Fixed-Window MLP Language Models

In the second part of the homework, we will build, train, and evaluate fixed-window MLP language models as we learned in class: given the context words in the fixed-size window on the left hand side, predict the next word continuation.



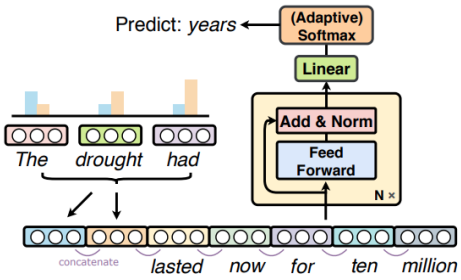
Fixed-window LM

Data Loading and Preprocessing

We will be using the same WikiText dataset we used for building n-gram LM in homework 2. We follow similar preprocessing steps to split paragraphs into sentences followed by tokenization (Now we have learned more about subword tokenization in class and the previous question!).
TODOs read the `preprocess_data` function in `ngram_lm.py` and complete the missing lines to prepare input-output pairs for training fixed-window LMs.
Hint: follow the requirements in the code comments.

Build our LM

Let's now turn to building our model. Here, we are following the footsteps of the neural probabilistic language model (NPLM) , which is extending earlier studies such as .



NPLM Architecture

We will follow a modular design for our implementation to make it more interpretable. In particular, will implement 3 layers:

- The **input layer** which loops up word embeddings, concatenates them, and transforms them into a hidden representation.
- The **middle layer** transforms the representation with a non-linearity.
- The **output layer** which transforms a hidden vector to a probability distribution over the words

Let's get to work!
TODOs: read and complete the `forward` functions for the following classes

- `NPLM_first_block`: input layer that embeds the input token ids into embedding vectors, concatenates the embeddings, applies linear transformation, layer normalization, and dropout
- `NPLM_block`: middle layers with linear transformation, tanh activation, residual connection, layer normalization, and dropout.

- `NPLM_final_block`: output layer that transforms hidden representation to log probability over the vocabulary with log softmax.

Hint: check out nn.LayerNorm and nn.Dropout, torch.tanh and nn.functional.log_softmax for more details on these layers and operations. For embedding concatenation, you may find torch.Tensor.view useful. Also, follow the step-by-step comments in the code for the requirements of the implementation.

TODOs: read and complete the `__init__` and `forward` functions for the `NPLM` class, which is the final model that stacks all the above layers.
Hint: remember to apply ReLU non-linear activation after each middle layer, details at nn.functional.relu.

Train and Evaluate the LM

Now it's time to train and evaluate it! Like the previous homework, we will use cross-entropy loss between the predictions of our language model and actual words that appeared in our training data. Note that we applied log softmax to the final output of the LM, as described in homework to, we will use negative log-likelihood loss as the training criterion to calculate the cross-entropy loss.

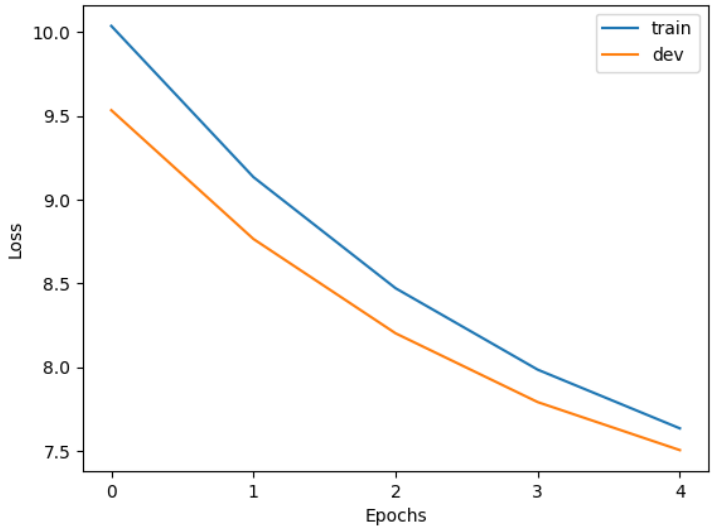
As introduced in the class, we will use **Perplexity** to evaluate our LM, as a measure of predictive quality of a language model.

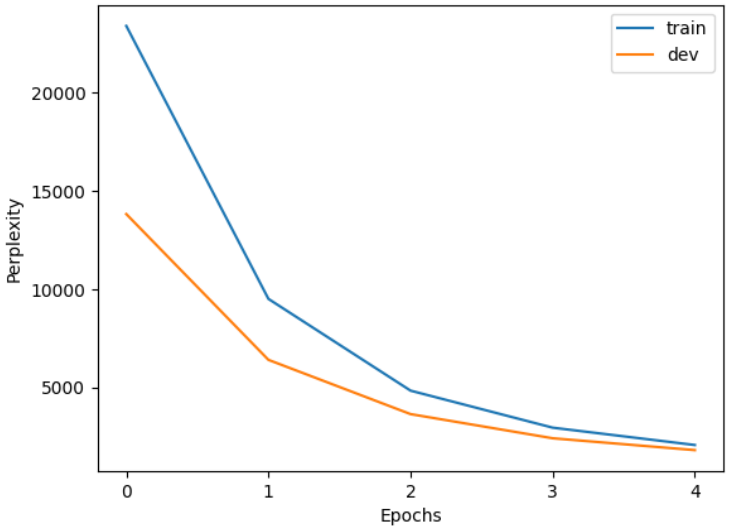
TODOs Read and complete the missing lines in `train` and `evaluate` functions in `mlp_lm.py` to calculate the perplexity.
Hint: remember in the class we discussed the connection between perplexity and cross-entropy loss, and note that in our implementation we took natural logarithm instead of the base of 2.

Note we are using Adam **(Adaptive Moment Estimation)** , which is a popular optimization algorithm used in deep learning and machine learning. It is a stochastic gradient descent (SGD) optimization algorithm that is well suited for training deep neural networks. The algorithm has some internal estimates to dynamically adjust the learning rates for each parameter based on its past gradients, which can result in faster convergence and improved performance compared to traditional SGD.

TODOs Once you finished all the implementations, run `load_data_mlp_lm` and `single_run_mlp_lm` in `main.py` to train and evaluate the model and paste the plots of loss and Perplexity here.

I trained with the provided configuration: embedding size 128, hidden size 512, 2 middle blocks, dropout 0.2, a window of 6 tokens, batch size 4096, learning rate 2e-6, and 5 epochs on the last 100,000 WikiText-103 training paragraphs (19,479,552 parameters).





Both curves go down every epoch. Training loss drops from about 10.0 to 7.6 and dev loss drops from 9.53 to 7.50, which brings dev perplexity down from 13,835 to 1,817. Dev loss staying slightly below training loss is not necessarily a problem. The training value is averaged over each epoch while the model is still changing, and in the first epoch dropout is also active during training but not during evaluation. The provided train loop never switches the model back to training mode after the first dev evaluation, so dropout is only on for epoch 0. The model is still improving after five epochs, but it is still well behind the provided pretrained model at 250.7 perplexity.

Sample From a Pre-trained LM

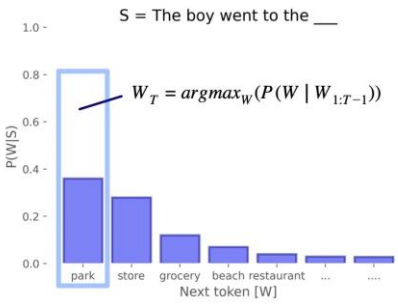
Note that compared with the original NPLM implementation, we reduce the model size by shrinking both the width and depth of our model, and we only use a small subset of the data. It is expected to take an hour or so to train the model on the CPU of your PC. Training the above LM with full size model and data might take hours to days. To save you time, we have trained a language model for you to play with. All you have to do is to download its weight parameters. Download the pre-trained weights and copy it to your local directory under `/hw4/`.

Now that we have the model weight downloaded, we can instantiate a model with these parameters. This will work in two steps.

- First we will need to create a new model (with potentially random weights).
- Then, we will copy the parameter values to the model using load_state_dict function.

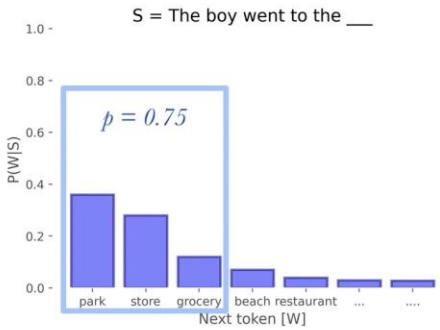
We then evaluate the loaded model on the dev set. Now that we have loaded a pre-trained LM, how can we sample from it? We will implement two strategies, greedy decoding and top-p sampling that we briefly discussed in homework 2.

For greedy, we select the most probable words (argmax).



Greedy Decoding

For top-p, we sample from the distribution proportional to word probabilities. Words with higher probabilities will be more likely to be sampled. However, we also have an option of filtering the low-probability tokens, and instead retaining tokens that constitute `top_p` probability.



Greedy Decoding

Read `generate_text` and `sample_from_mlp_lm` in `mlp_lm.py` for more details about how to load the pre-trained model, evaluate on dev set and perform greedy/top-p sampling. You can learn more about top-p sampling implementation at TopPLogitsWarper.

TODOs Run the `sample_from_trained_mlp_lm` in `main.py` to sample from the pre-trained LM, paste the completion to the prefix under different sampling strategies here, and describe in 2-3 sentences your findings.
Hint: compare the output of the above generations, which one is your favorite? Explain why this choice of sampling leads to better text generation.

Dev set with the pre-trained model: Perplexity 250.67, Loss 5.52.

Greedy: The best perks of living on the east side of the city. ". "... ".... ". "... ".... ". "

Sampling with p = 0.0: The best perks of living on the east side of the city. ". "... ".... ". "... ".... ". "

Sampling with p = 0.3: The best perks of living on the east of the U. S. government had a year. ". ", the episode ' s very much of the most common age. "..

Sampling with p = 1.0: The best perks of living on the east slopes of Mexico and lip Germany was once over the 20th century but none of Rose concluded children, adopted with the frame of the magnificent Age by trees.

Greedy and p = 0.0 give exactly the same text in this implementation because the top-p sampler ends up keeping only the highest-probability token. It starts out well ("on the east side of the city.") but then gets stuck repeating quotes and periods. p = 1.0 is the most varied but drifts into nonsense. My favorite is p = 0.3 because it avoids the repetition loop while mostly sticking to likely tokens, although with only a 6-token window none of the outputs stay on topic for long.

Optional Feedback

Have feedback for this assignment? Found something confusing? We'd love to hear from you!